

PROGRAMAÇÃO EM HASKELL



Capítulo 5 - Compreensão de Listas

Compreensão de conjuntos

Na matemática, a notação de **compreensão** é usada para construir novos conjuntos a partir de conjuntos existentes

$$\{x^2 \mid x \in \{1 \dots 5\}\}$$

O conjunto $\{1, 4, 9, 16, 25\}$ de todos os números x^2 tal que x é um elemento do conjunto $\{1 \dots 5\}$

Compreensão de listas

Em Haskell, uma notação de compreensão semelhante pode ser usada para construir **novas listas** a partir de **listas existentes**

```
[x^2 | x ← [1..5]]
```

A lista **[1, 4, 9, 16, 25]** de todos os números **x^2** tal que **x** é um elemento da lista **[1..5]**

Nota

- A expressão $x \leftarrow [1..5]$ é chamada de **gerador**, pois indica como gerar valores para x , sendo $\leftarrow$ escrito como $<-$
- As compreensões podem ter **múltiplos geradores**, separados por vírgulas, por exemplo

```
> [(x,y) | x <- [1,2,3], y <- [4,5]]  
[(1,4), (1,5), (2,4), (2,5), (3,4), (3,5)]
```

- Alterar a **ordem** dos geradores altera a ordem dos elementos na lista final

```
> [(x,y) | y ← [4,5], x ← [1,2,3]]
```

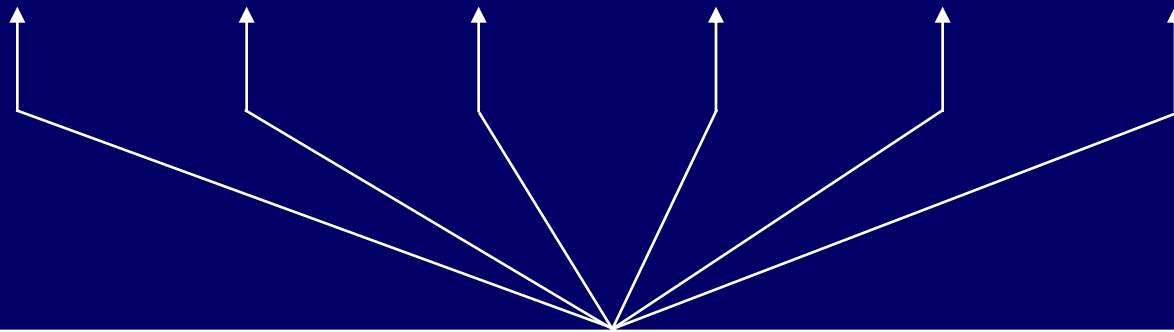
```
[(1,4), (2,4), (3,4), (1,5), (2,5), (3,5)]
```

- Geradores múltiplos são como **loops aninhados**, com geradores posteriores como loops aninhados mais internamente, cujas variáveis mudam de valor com mais frequência

- Por exemplo

> $[(x,y) \mid y \leftarrow [4,5], x \leftarrow [1,2,3]]$

$[(1,4), (2,4), (3,4), (1,5), (2,5), (3,5)]$



$x \leftarrow [1,2,3]$ é o último gerador,
portanto o valor do componente x de
cada par muda com mais frequência

Geradores dependentes

Geradores posteriores podem **depend**er das variáveis que são introduzidas por geradores anteriores

$$[(x, y) \mid x \leftarrow [1..3], y \leftarrow [x..3]]$$

A lista $[(1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]$ de todos os pares de números (x, y) tais que x, y são elementos da lista $[1..3]$ e $y \geq x$

Usando um gerador dependente, pode-se definir a função de biblioteca que concatena uma lista de listas

```
concat :: [[a]] → [a]  
concat xss = [x | xs ← xss, x ← xs]
```

Por exemplo

```
> concat [[1,2,3],[4,5],[6]]  
[1,2,3,4,5,6]
```


Guards

Compreensão de listas pode usar **guards** (protetores, filtro) para restringir os valores produzidos por geradores anteriores

```
[x | x ← [1..10], even x]
```

A lista **[2, 4, 6, 8, 10]** de todos os números **x** tal que **x** é um elemento da lista **[1..10]** e **x** é par

Usando *guards*, pode-se definir uma função que mapeia um inteiro positivo para sua **lista de fatores**

```
factors :: Int → [Int]
factors n =
    [x | x ← [1..n], n `mod` x == 0]
```

Por exemplo

```
> factors 15
[1, 3, 5, 15]
```

Um inteiro positivo é **primo** se seus únicos fatores são **1** e ele mesmo. Assim, usando fatores, pode-se definir uma função que decide se um número é primo

```
prime :: Int → Bool  
prime n = factors n == [1,n]
```

Por exemplo

```
> prime 15  
False  
  
> prime 7  
True
```

Usando *guards*, pode-se definir uma função que retorna a lista de **todos os primos** até um determinado limite

```
primes :: Int → [Int]
primes n = [x | x ← [2..n], prime x]
```

Por exemplo

```
> primes 40
[2,3,5,7,11,13,17,19,23,29,31,37]
```

A função Zip

Uma função de biblioteca útil é **zip**, que mapeia duas listas para uma lista de pares de seus elementos correspondentes

```
zip :: [a] → [b] → [(a,b)]
```

Por exemplo

```
> zip ['a', 'b', 'c'] [1,2,3,4]  
[('a',1), ('b',2), ('c',3)]
```

Usando **zip**, pode-se definir uma função que retorna a lista de **todos os pares** de elementos adjacentes de uma lista

```
pairs :: [a] → [(a,a)]  
pairs xs = zip xs (tail xs)
```

Por exemplo

```
> pairs [1,2,3,4]  
[(1,2), (2,3), (3,4)]
```

Usando **pairs**, pode-se definir uma função que decide se os elementos em uma lista estão **ordenados**

```
sorted :: Ord a => [a] -> Bool  
sorted xs = and [x ≤ y | (x,y) ← pairs xs]
```

Por exemplo

```
> sorted [1,2,3,4]  
True  
  
> sorted [1,3,2,4]  
False
```

Usando **zip** podemos definir uma função que retorna a lista de **todas as posições** de um valor em uma lista

```
positions :: Eq a => a -> [a] -> [Int]
positions x xs =
    [i | (x',i) <- zip xs [0..], x == x']
```

Por exemplo

```
> positions 0 [1,0,0,1,0,1,1,0]
[1,2,4,7]
```


Compreensão de strings

Uma **string** é uma sequência de caracteres entre aspas duplas. Internamente, no entanto, as strings são representadas como listas de caracteres.

`"abc" :: String`



Significa `['a', 'b', 'c'] :: [Char]`

Como strings são apenas tipos especiais de listas, qualquer **função polimórfica** que opera em listas pode ser aplicada a strings, por exemplo

```
> length "abcde"
```

```
5
```

```
> take 3 "abcde"
```

```
"abc"
```

```
> zip "abc" [1,2,3,4]
```

```
[('a',1),('b',2),('c',3)]
```

Da mesma forma, compreensão de lista pode ser usada para definir funções em strings, como contar quantas vezes um caractere ocorre em uma cadeia de caracteres

```
count :: Char → String → Int  
count x xs = length [x' | x' ← xs, x == x']
```

Por exemplo

```
> count 's' "Mississippi"  
4
```

Exercícios

- (1) Usando compreensão de listas, faça uma expressão que calcule a soma dos quadrados dos primeiros cem inteiros $1^2 + 2^2 + \dots 100^2$
- (2) Suponha que um *grid* de coordenadas de tamanho $m \times n$ é dado pela lista de todos os pares (x,y) de inteiros tal que $0 \leq x \leq m$ e $0 \leq y \leq n$. Usando compreensão de listas, defina uma função

```
grid :: Int → Int → [(Int,Int)]
```

que retorne um *grid* de um dado tamanho, e.g.

```
> grid 1 2  
[(0,0), (0,1), (0,2), (1,0), (1,1), (1,2)]
```

Exercícios

(3) Usando compreensão de listas e a função `grid` anterior, defina a função

```
square :: Int → [(Int,Int)]
```

que retorne uma coordenada quadrada de tamanho `n`, excluindo a diagonal de `(0,0)` a `(n,n)`. Por exemplo

```
> square 2  
[(0,1),(0,2),(1,0),(1,2),(2,0),(2,1)]
```

Exercícios

- (4) De maneira semelhante à função `length`, mostre como a função de biblioteca

```
replicate :: Int → a → [a]
```

que produz uma lista de elementos idênticos pode ser definida usando compreensão de lista, por exemplo

```
> replicate 3 True  
[True,True,True]
```

Exercícios

- (5) Uma tripla (x, y, z) de inteiros positivos é chamada **pitagórica** se $x^2 + y^2 = z^2$. Usando compreensão de lista, defina uma função

```
pyths :: Int → [(Int,Int,Int)]
```

que mapeia um inteiro n para todas essas triplas com componentes em $[1..n]$, por exemplo

```
> pyths 5  
[(3,4,5),(4,3,5)]
```

(6) Um inteiro positivo é **perfeito** se for igual à soma de todos os seus fatores, excluindo o próprio número. Usando compreensão de lista, defina uma função

```
perfects :: Int → [Int]
```

que retorna a lista de todos os números perfeitos até um determinado limite, por exemplo

```
> perfects 500
```

```
[6, 28, 496]
```


- (7) Mostre como a compreensão da lista `[(x, y) | x <- [1,2], y <- [3,4]]` com dois geradores pode ser reexpressa usando duas compreensões com geradores únicos. Dica: aninhe uma compreensão dentro da outra e faça uso da função de biblioteca `concat :: [[a]] -> [a]`
- (8) Redefina a função `positions` usando a função `find`

- (9) O **produto escalar** de duas listas de inteiros **xs** e **ys** de comprimento **n** é dado pela soma dos produtos dos inteiros correspondentes

$$\sum_{i=0}^{n-1} (xs_i * ys_i)$$

Usando compreensão de lista, defina uma função que retorne o produto escalar de duas listas

(10) Modifique o programa de cifra de César para lidar também com letras maiúsculas